In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, RepeatVector, TimeDistributed, Dense
```

In [3]:
```
# Load dataset
df = pd.read_csv("energy consumption.csv")

print(df.head())
print(df.shape)

# Preprocess
df["Timestamp"] = pd.to_datetime(df["Timestamp"], dayfirst=True)
df = df.sort_values("Timestamp").reset_index(drop=True)
df = df.fillna(df.median(numeric_only=True))

X = df.drop(["Timestamp", "Anomaly_Label"], axis=1)
y = df["Anomaly_Label"]
```

```
          Timestamp  Electricity_Consumed  Temperature  Humidity  Wind_Speed  \
0  01-01-2024 00:00              0.457786     0.469524  0.396368    0.445441
1  01-01-2024 00:30              0.351956     0.465545  0.451184    0.458729
2  01-01-2024 01:00              0.482948     0.285415  0.408289    0.470360
3  01-01-2024 01:30              0.628838     0.482095  0.512308    0.576241
4  01-01-2024 02:00              0.335974     0.624741  0.672021    0.373004

   Avg_Past_Consumption  Anomaly_Label
0              0.692057              0
1              0.539874              0
2              0.614724              0
3              0.757044              0
4              0.673981              0
(5000, 7)
```

In [4]:
```
# Create time windows
window_size = 24

X_seq = []
y_seq = []

for i in range(len(X) - window_size + 1):
    X_seq.append(X.iloc[i:i + window_size].values)
    y_seq.append(int(np.any(y.iloc[i:i + window_size])))

X_seq = np.array(X_seq)
y_seq = np.array(y_seq)
```

In [18]:
```
# Split windows and labels using the same split
split = int(0.8 * len(X_seq))

x_train = X_seq[:split]
x_test = X_seq[split:]
y_train = y_seq[:split]
y_test = y_seq[split:]

# Keep only normal training windows
x_train = x_train[y_train == 0]
```

In [19]:
```
# Normalize using training data only
scaler = MinMaxScaler()
scaler.fit(x_train.reshape(-1, x_train.shape[2]))

x_train = scaler.transform(
    x_train.reshape(-1, x_train.shape[2])
).reshape(x_train.shape)

x_test = scaler.transform(
    x_test.reshape(-1, x_test.shape[2])
).reshape(x_test.shape)
```

In [20]:
```
# Build LSTM Autoencoder
model = Sequential([
    LSTM(32, activation="relu", input_shape=(24, x_train.shape[2])),
```

```
    RepeatVector(24),
    LSTM(32, activation="relu", return_sequences=True),
    TimeDistributed(Dense(x_train.shape[2]))
])

model.compile(optimizer="adam", loss="mse")
```

```
C:\Users\ombah\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `inp
ut_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first lay
er in the model instead.
  super().__init__(**kwargs)
```

In [21]:
```
# Train model
history = model.fit(
    x_train, x_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

```
Epoch 1/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - loss: 0.1147 - val_loss: 0.0421
Epoch 2/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0401 - val_loss: 0.0332
Epoch 3/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0324 - val_loss: 0.0278
Epoch 4/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - loss: 0.0269 - val_loss: 0.0248
Epoch 5/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0251 - val_loss: 0.0240
Epoch 6/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0247 - val_loss: 0.0239
Epoch 7/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.0245 - val_loss: 0.0238
Epoch 8/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0245 - val_loss: 0.0237
Epoch 9/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.0244 - val_loss: 0.0238
Epoch 10/10
51/51 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0243 - val_loss: 0.0235
```

In [22]:
```
# Reconstruction error on normal training data
train_pred = model.predict(x_train, verbose=0)

train_error = np.mean(
    np.square(x_train - train_pred),
    axis=(1, 2)
)

# Set anomaly threshold
threshold = train_error.mean() + 3 * train_error.std()

print("Anomaly Threshold:", threshold)
```

```
Anomaly Threshold: 0.0362800210534109
```

In [23]:
```
# Reconstruction error on test data
test_pred = model.predict(x_test, verbose=0)

test_error = np.mean(
    np.square(x_test - test_pred),
    axis=(1, 2)
)

# Predict anomalies
y_pred = (test_error > threshold).astype(int)

print("Detected Anomalies:", np.sum(y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Detected Anomalies: 41

Confusion Matrix:
[[484   0]
 [471  41]]

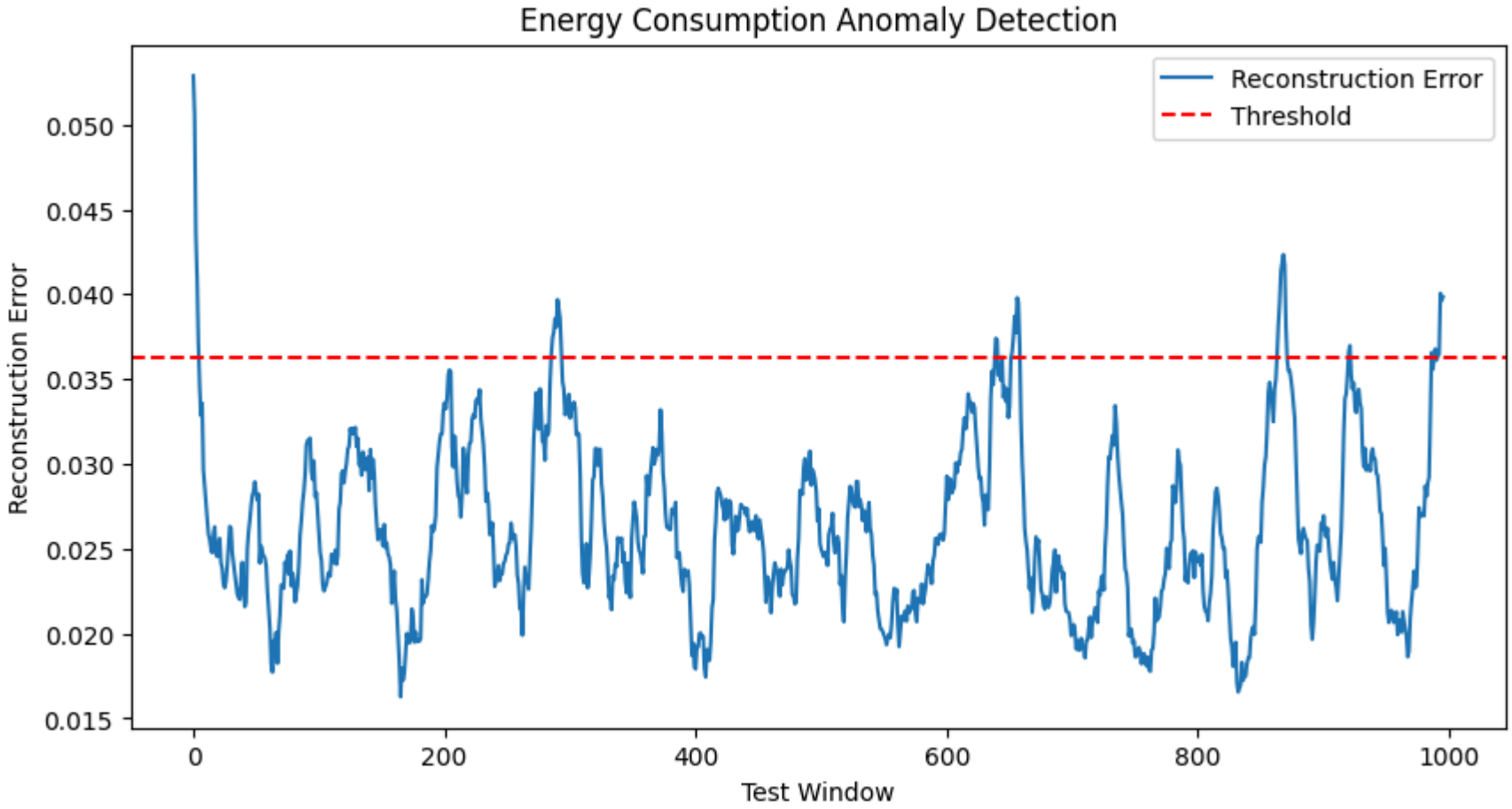
Classification Report:
              precision    recall  f1-score   support

           0       0.51      1.00      0.67       484
           1       1.00      0.08      0.15       512

    accuracy                           0.53       996
   macro avg       0.75      0.54      0.41       996
weighted avg       0.76      0.53      0.40       996
```

In [24]:
```
plt.figure(figsize=(10, 5))
plt.plot(test_error, label="Reconstruction Error")
plt.axhline(threshold, color="red", linestyle="--", label="Threshold")

plt.xlabel("Test Window")
plt.ylabel("Reconstruction Error")
plt.title("Energy Consumption Anomaly Detection")
plt.legend()
plt.show()
```


Energy Consumption Anomaly Detection
Reconstruction Error
Threshold
Reconstruction Error
0.050
0.045
0.040
0.035
0.030
0.025
0.020
0.015
0
200
400
600
800
1000
Test Window

In [ ]: